第 2 章　无线局域网视频传输仿真

本章重点阐述无线局域网视频传输原理，包含视频编解码基本概念、IEEE 802.11e 协议基本原理以及常见的视频评价标准，旨在让读者了解 H.264 编码工具使用，以及 IEEE 802.11e 协议对于视频传输的影响，并能用 NS-2 工具进行分析。

2.1 预备知识

2.1.1 H.264 视频编解码

H.264 视频编码标准由 ITU-T 和 MPEG 共同推出，自发布之日起就在业界产生了巨大影响。事实上 H.264 标准属于 MPEG-4 家族一部分，即 MPEG-4 ISO-14496 第 10 部分，又称 MPEG-4/AVC。同 MPEG-4 重点考虑的灵活性和交互性不同，H.264 采用多项提高图像质量和增加压缩比的技术，着重强调更高编码压缩率和传输可靠性，在数字电视广播、实时视频通信、网络流媒体等领域具有广泛的应用。H.264 编码更加节省码流，不仅比 MPEG-4 节约了 50%的码率，还具有较强的抗误码性，可适应丢包率高、干扰严重的无线信道中的视频传输，获得平稳的图像质量。H.264 标准使运动图像压缩技术上升到一个更高的阶段，在较低带宽上提供高质量图像传输是 H.264 应用亮点，很好适应了国内运营商接入网带宽非常有限的状况。在编码框架方面，H.264 仍采用与前期标准类似的结构，即块结构的混合编码框架，主要结构如下图：

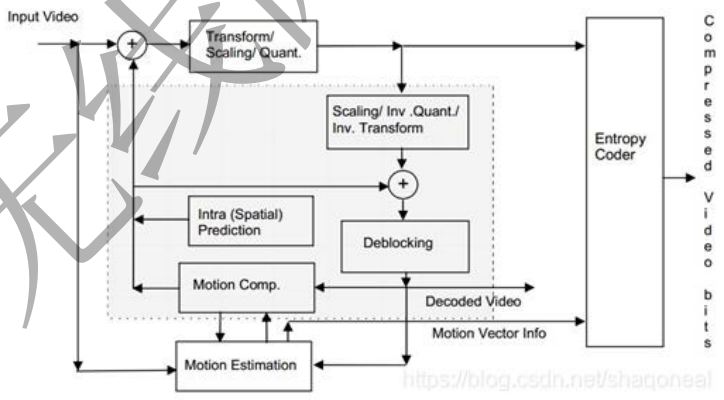


图 2.1　H.264 编码架构示意图

H.264 编码过程中，每帧图像被分为 1 或多个条带进行编码，每个条带包含多个宏块。宏块是 H.264 基本编码单元，其基本结构含一个 16×16 亮度像素块和两个 8×8 色度像素块，以及其他宏块头信息。在对宏块进行编码时，每个宏块会分割成多种不同大小子块进行预测。帧内预测采用的块大小可能为 16×16 或者 4×4，帧间预测/运动补偿采用的块可能有 7 种不同的形状：16×16、16×8、8×16、8×8、8×4、4×8 和 4×4。相比早期标准只能按照宏块或者半个宏

块进行运动补偿，H.264 采用的这种更加细分的宏块分割方法提供了更高预测精度和编码效率。在变换编码方面，针对预测残差数据进行的变换块大小为 4×4 或 8×8。相比于仅支持 8×8 大小的变换块的早期版本，H.264 避免了变换/逆变换中经常出现的失配问题。

同前期标准类似，H.264 条带也有不同类型，其中最常用的有 I 条带（帧内编码条带只包含 I 宏块）、P 条带(单向帧间编码条带，可能包含 P 宏块和 I 宏块)和 B 条带（双向帧间编码条带，可能包含 B 宏块和 I 宏块）等。另外，为支持码流切换，在扩展档次中还定义了 SI 和 SP 片。视频编码工具，如预测编码、变化量化、熵编码等，主要工作在条带层或以下，该层常被称为视频编码层(Video Coding Layer, VCL)。在条带层以上的数据和算法常称为网络抽象层(Network Abstraction Layer, NAL)，其意义在于提升 H.264 格式视频对网络传输和数据存储的亲和性。有关 H.264 编码，本书不做深入讨论，感兴趣的读者请查阅相关文献资料。

2.1.2 IEEE 802.11e 协议

IEEE 802.11e 标准定义了无线局域网 MAC 层服务质量，支持语音、视频等多媒体业务在无线局域网中的应用。该标准扩展了原 IEEE 802.11 MAC 层 DCF 和 PCF 信道接入机制，增加了混合协调功能(Hybrid Coordination Function, HCF)，形成了增强型分布式信道访问(Enhanced Distributed Channel Access, EDCA)和 HCF 控制的信道访问(HCF Controlled Channel Access, HCCA)接入规范。前者增强了 DCF 机制，区分不同业务的优先级，保障高优先级业务的信道接入能力，并在一定程度上保障了高优先级业务的带宽。后者增强了 PCF 机制，通过 QAP 的集中控制，以轮询方式为 QSTA 分配空口资源，提供改善的访问带宽并且减少了高优先级业务的延迟。由于 HCCA 机制相比 EDCA、PCF 机制更加复杂，目前被各厂商束之高阁，没有得到推广，本书仿真实验将考虑 EDCA。图 2.2 是 EDCA 的示意图，其表明 802.11e 可以提供 4 个不同的优先级，也可称为接入类别(Access Categories, AC)，从高到低排序分别是：

①语音服务(Voice, AC_VO)：一般为 VoIP 流量，对延迟最为敏感，同时也是优先级最高的流量。

②视频服务(Video, AC_VI)：视频流量的优先级低于语音服务，高于其他两项。视频服务也是延迟敏感类型的服务，所以具有一定的优先级。

③尽力传输(Best-effort, AC_BE)：默认的无线流量类型就是 Best-effort 类型，比如网页访问的数据流量类型。对于延迟有一定需求，但是没有那么敏感。

④背景流量(Background, AC_BK)：对延迟要求最不敏感的流量，比如文件传输，打印作业的流量。

无线网络技术

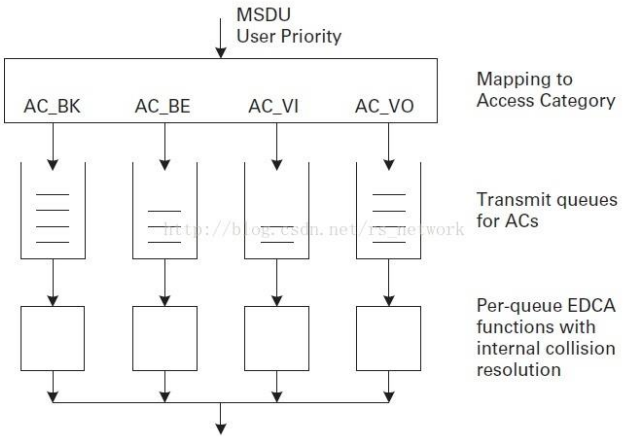


图 2.2　EDCA 示意图

为支持 EDCA 接入类别优先级，IEEE 802.11e 标准定义了竞争窗口(Contention Window, CW)更新，并给出了 CW 更新的边界值 CWmin 和 CWmax，具体如下表：

表 2.1　CW 边界计算

访问类别(AC)	CWmin	CWmax
Background (AC_BK)	aCWmin	aCWmax
Best Effort (AC_BE)	aCWmin	aCWmax
Video (AC_VI)	(aCWmin+1)/2-1	aCWmin
Voice (AC_VO)	(aCWmin+1)/4-1	(aCWmin+1)/2-1

计算遵循的步骤为：①确定接入类别：每个类别对应不同流量类型和优先级，如语音、视频等；②选择物理层：不同物理层传输速率和媒介特性不同，可能需不同设置 CW；③定义 aCWmin 和 aCWmax：物理层不同，基值 aCWmin 和 aCWmax 不同。

除信道接入机制增强，IEEE 802.11e 还引入了 BLOCK ACK、DLS、No-ACK 等技术，有效提高了无线信道带宽和吞吐量。自 2005 年 IEEE 802.11e 标准化后，至今仍未完全商用，芯片厂商仅支持其中少量特性。在后续的 802.11 协议设计中，802.11e 的主要设计得到了应用：

①802.11n 在 802.11e 的 TXOP 和 Block ACK 基础上，进一步演进协议的 MAC 层效率，以及节能模式上，比如 PSMP，SMPS 之类的引入；

②802.11s 规范了 Mesh 网络下 802.11 的工作原理，其采用的多信道 MAC 层接入方式则是基于 802.11e 的 EDCA；

③802.11z 继续深化 802.11e 中的直接链路设置，其与通信网所提的 D2D 概念相同，且应用广泛；

④802.11ae 在 802.11e 基础上规范了 802.11 协议控制帧，完善了 802.11 协议的 QoS 体系。

2.1.3 视频质量评价指标

视频质量评价分主观和客观两类，其中主观评价由人(肉眼)观看后对视频质量进行评价(评分)，而客观评价则采用统计学的方法评价视频的质量。为了对无线局域网视频传输性能进行分析，本书采用客观指标**峰值信噪比**（Peak Signal to Noise Ratio, PSNR）进行评价。

在无线局域网中，链路存在随机丢包且部分视频传输采用 UDP 协议，而 UDP 无可靠性保证，致使接收端收到的视频帧不完整。为表征无线局域网视频传输性能，采用如下公式进行计算：

$$PSNR = 10 * log_{10}\frac{(2^n-1)^2}{MSE}$$

其中，*n* 表示编码时采用的 bit 深度，如 8bit 情况下 n 为 8；MSE表示收到的视频与源视频的均方差，具体定义如下：

$$MSE = \frac{1}{m \cdot n}\sum_{m-1}^{i=0}\sum_{n-1}^{j=0}[I(i,j) - K(i,j)]^2$$

方差表示波动性，方差越小则波动越小、越稳定。PSNR 计算公式将方差作为分母，再取对数，因而 PSNR 的值越高越好（表明接收到的视频质量越好），单位为 dB。通常，PSNR 高于 40dB 表示画质极好（接近原始视频）；PSNR 在 30dB~40dB 之间表示画质较好（有失真但可接受）；PSNR 在 20dB~30dB 之间表示画质差；PSNR 小于 20db 表示画质不可接受。

2.2 实验环境

EvalVid 框架由 Jirka Klaue 等在 2003 年提出，受到广泛关注（Google 引用超过 1000 次），其提供了完整的框架和相关工具集，以便于提供统一的视频传输质量的评估。EvalVid 具有模块化结构的特点，可根据实际需要替换底层传输协议、应用层视频编/解码器，支持不同格式的视频。该框架可用于实测和仿真环境，且所有工具实现都采用 C 语言。图 2.3 为 EvalVid 的工作流程示意图，更多详细内容可查阅文献。

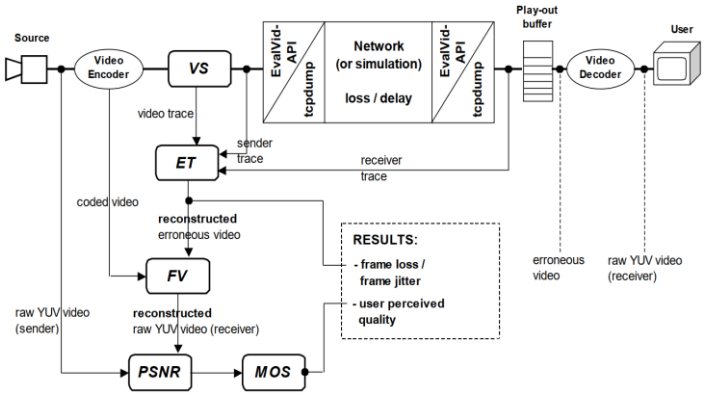


图 2.3　EvalVid 框架工作流

无线网络技术

2.3 实验步骤

2.3.1 扩展 NS-2 仿真工具

为支持视频传输仿真，NS-2 需扩展应用层(文件夹为“**myevalvid**”)和 MAC 层模块(文件夹为“**80211e**”)，模块代码可从本书提供的资源中获得。请将上述文件夹整体拷贝至“**/home/wiotlab/WIoTLab/tools/ns2/ns-2.35**”，并按以下步骤修改 NS-2 源码，所有修改都为“**/home/wiotlab/WIoTLab/tools/ns2/ns-2.35**”路径下的文件。

(1)应用层扩展

①修改“**common/packet.h**”文件，增加以下加粗部分的内容，可以搜索“**int aomdv_salvage_count_**;”快速定位修改位置：

```
struct hdr_cmn {
    ……
    //Monarch extn begins
    nsaddr_t prev_hop_;// IP addr of forwarding hop
    nsaddr_t next_hop_;// next hop for this packet
    int addr_type_;// type of next_hop_ addr
    nsaddr_t last_hop_;// for tracing on multi-user channels
    // AOMDV patch
    int aomdv_salvage_count_;
    int frametype_; // frame type for MPEG video transmission
    double sendtime_; // send time
    unsigned long int frame_pkt_id_;
……
```

②修改“**common/agent.h**”文件，增加以下加粗部分的内容，可以搜索“**inline packet_t get_pkttype()**”快速定位修改位置：

```
class Agent: public Connector {
public:
    Agent(packet_t pktType);
    virtual ~Agent();
    ……
    void set_pkttype(packet_t pkttype) { type_ = pkttype; }
    inline packet_t get_pkttype() { return type_; }
    inline void set_frametype(int type) { frametype_ = type; }
    inline void set_prio(int prio) { prio_ = prio; }
    protected:
    ……
    int prio_;// for IPv6 prio field
    int flags_;// for experiments (see ip.h)
    int defttl_;// default ttl for outgoing pkts
```

无线网络技术

```
    int frametype_; // frame type for MPEG video transmission
    ……
```

③修改“**common/agent.cc**”文件，增加以下加粗部分的内容，可以搜索“**ch->error() = 0**”快速定位修改位置：

```
Agent::Agent(packet_t pkttype):
    size_(0), type_(pkttype),
    channel_(0), traceName_(NULL), frametype_(0),
    oldValueList_(NULL), app_(0), et_(0)
{
}
……
Agent::initpkt(Packet* p) const
{
    ……
    ch->error() = 0;/* pkt not corrupt to start with */
    ch->frametype_= frametype_;
    ……
```

④修改“tcl/lib/ns-default.tcl”文件，在其最后增加以下内容：

```
Agent/my_UDP set packetSize_ 1000
Tracefile set debug_ 0
```

⑤修改“**Makefile.in**”文件，增加以下加粗部分的内容，可以搜索“**apps/pbc.o**”快速定位修改位置：

```
……
wpan/p802_15_4csmaca.o wpan/p802_15_4fail.o \
wpan/p802_15_4hlist.o wpan/p802_15_4mac.o \
wpan/p802_15_4nam.o wpan/p802_15_4phy.o \
wpan/p802_15_4sscs.o wpan/p802_15_4timer.o \
wpan/p802_15_4trace.o wpan/p802_15_4transac.o \
apps/pbc.o \
myevalvid/my_udp.o myevalvid/myevalvid_sink.o myevalvid/myevalvid.o \
……
```

上述文件修改后，需重新编译，切换至“ns-2.35”的同级目录，然后执行“**./install**”重新安装。

(2)扩展 802.11e 协议

①修改“**common/packet.h**”文件，增加以下加粗部分的内容，可以搜索“**#define HDR_LMS(p)**”快速定位修改位置：

```
……
#define HDR_LMS(p) (hdr_lms::access(p))
#define HDR_MAC802_11E(p) ((hdr_mac802_11e *)hdr_mac::access(p))
```

无线网络技术

……

②修改“**mac/wireless-phy.h**”文件，更新以下加粗部分的内容，可以搜索“**MobileNode* node_;**”快速定位修改位置：

……
// MobileNode* node_;// Mobile Node to which interface is attached
enum ChannelStatus { SLEEP, IDLE, RECV, SEND, RECVING, SENDING};
……

③修改“tcl/lib/ns-default.tcl”文件，在其最后增加以下内容：

```
Queue/DTail set drop_front_ false
Queue/DTail set summarystats_ false
Queue/DTail set queue_in_bytes_ false
Queue/DTail set mean_pktsize_ 500
Queue/DTail/PriQ set Prefer_Routing_Protocols 1
Queue/DTail/PriQ set Max_Levels 4
Queue/DTail/PriQ set Levels 4
Mac/802_11e set SlotTime_ 0.000020;# 20us
Mac/802_11e set SIFS_ 0.000010;# 10us
Mac/802_11e set PreambleLength_ 144;#144 bit
Mac/802_11e set PLCPHeaderLength_ 48;# 48 bits
Mac/802_11e set PLCPDataRate_ 1.0e6;# 1Mbps
Mac/802_11e set RTSThreshold_ 3000 ;# bytes
Mac/802_11e set ShortRetryLimit_ 7;# retransmittions
Mac/802_11e set LongRetryLimit_ 4;# retransmissions
```

④修改“tcl/lan/ns-mac.tcl”文件，增加以下加粗部分的内容，可以搜索“**Mac/Multihop set hlen_ 16**”快速定位修改位置：

```
        ……
        Mac/Multihop set hlen_ 16
}
if [TclObject is-class Mac/802_11e] {
        Mac/802_11e set delay_ 64us
        Mac/802_11e set ifs_ 16us
        Mac/802_11e set slotTime_ 16us
        Mac/802_11e set cwmin_ 16
        Mac/802_11e set cwmax_ 1024
        Mac/802_11e set rtxLimit_ 16
        Mac/802_11e set bssId_ -1
        Mac/802_11e set sifs_ 8us
        Mac/802_11e set pifs_ 12us
        Mac/802_11e set difs_ 16us
        Mac/802_11e set rtxAckLimit_ 1
        Mac/802_11e set rtxRtsLimit_ 3
        Mac/802_11e set basicRate_ 1Mb    ;# set this to 0 if want to use bandwidth_ for
```

无线网络技术

```
    Mac/802_11e set dataRate_ 1Mb     ;# both control and data pkts
    Mac/802_11e set cfb_ 0 ;# disables CFB
}
……
```

⑤修改“tcl/lib/ns-lib.tcl”文件，增加以下加粗部分的内容，可以搜索“**source ../mobility/com.tcl**”快速定位修改位置：

```
……
source ../mobility/com.tcl
source ../../mac/802_11e/priority.tcl
……
```

⑥修改“tcl/lib/ns-mobilenode.tcl”文件，增加以下加粗部分的内容，可以搜索“**$ifq set limit_ $qlen**”快速定位修改位置：

```
……
$ifq set limit_ $qlen
if {$qtype == "Queue/DTail/PriQ"} {
    priority ifq
}
……
if {$mactype == "Mac/802_11"} {
    $mac nodes [$god_ num_nodes]
}
if {$mactype == "Mac/802_11e"} {
    $mac nodes [$god_ num_nodes]
}
……
```

⑦修改“**Makefile.in**”文件，增加以下加粗部分的内容，可以搜索“**-I./wpan**”和“**apps/pbc.o**”快速定位修改位置：

```
……
-I./wpan -I./802_11e
……
wpan/p802_15_4csmaca.o wpan/p802_15_4fail.o \
wpan/p802_15_4hlist.o wpan/p802_15_4mac.o \
wpan/p802_15_4nam.o wpan/p802_15_4phy.o \
wpan/p802_15_4sscs.o wpan/p802_15_4timer.o \
wpan/p802_15_4trace.o wpan/p802_15_4transac.o \
apps/pbc.o \
mac/802_11e/mac-802_11e.o mac/802_11e/priq.o \
mac/802_11e/d-tail.o mac/802_11e/mac-timers_802_11e.o \
……
tcl/lib/ns-qsnode.tcl \
mac/802_11e/priority.tcl \
```

无线网络技术

……

上述文件修改后，需重新编译，切换至“ns-2.35”的同级目录，然后执行“**./install**”重新安装。

2.3.2 视频处理流程

该实验采用 YUV 格式视频源[1]，在正式仿真实验前，需按如下步骤进行视频数据处理，以便能够被 NS-2 仿真器使用。所有视频处理工具见本书资源包。

①将 YUV 视频编码为 MPEG4 格式，使用 **xvid_encraw** 工具。假设视频的帧率为 30，文件名为 **foreman_qcif.yuv**，视频分辨率为 352*288。

```
$./xvid_encraw -i foreman_qcif.yuv -w 352 -h 288 -framerate 30 -max_key_interval 30 -o a01.m4v
```

②将 M4V 格式文件转为 MP4 文件，其中包含视频样本(帧)和一个提示轨道，使用 **MP4Box** 工具，具体如下：

```
$./MP4Box -hint -mtu 1024 -fps 30 -add a01.m4v a01.mp4
```

③使用 EvalVid 中的 **mp4trace** 工具生成仿真所需的数据文件，其中的 IP 地址和端口号可以任意，具体如下：

```
$./mp4trace -f -s 192.168.0.2 12346 a01.mp4 > st_a01
```

经过上述三步操作，生成了从 YUV 视频到 NS-2 仿真所需的数据，保存在上面的 **st_a01** 文件中。NS-2 仿真结束后，假设生成了 **sd_a01**(每个数据包发送时间)和 **rd_a01**(每个数据包接收时间)文件，其保存了仿真产生的视频相关的数据。

④根据上述数据，使用 **etmp4** 工具重建接收端接收到的视频，即删除原视频在传输过程中丢失和损坏的帧。

```
$./etmp4 -f -0 sd_a01 rd_a01 st_a01 a01.mp4 a01e
```

⑤使用 **ffmpeg** 工具将 MP4 视频转为 YUV 视频，并利用 **psnr** 工具计算原视频和目标视频的 PSNR 值，具体如下：

```
$ ffmpeg -i a01e.mp4 a01e.yuv
$./psnr 352 288 420 foreman_qcif.yuv a01e.yuv
```

2.3.3 视频传输仿真

(1)简单视频传输

该实验建立了简单的有线链路，包含发送和接收节点各 1 个、路由节点 2 个，具体代码如下所示。

```
set ns [new Simulator]
```

[1] http://trace.eas.asu.edu/yuv/index.html

无线网络技术

```
set nd [open lab2.tr w]
$ns trace-all $nd
set max_fragmented_size 1024
set packetSize 1052
set s1 [$ns node]
set r1 [$ns node]
set r2 [$ns node]
set d1 [$ns node]
$ns duplex-link $s1 $r1 10Mb 1ms DropTail
$ns simplex-link $r1 $r2 640kb 1ms DropTail
$ns simplex-link $r2 $r1 640Mb 1ms DropTail
$ns duplex-link $r2 $d1 10Mb 1ms DropTail
set qr1r2 [[$ns link $r1 $r2] queue]
$qr1r2 set limit_ 50
set udp1 [new Agent/my_UDP]
$ns attach-agent $s1 $udp1
$udp1 set packetSize_ $packetSize
$udp1 set_filename sd_a01
set null1 [new Agent/myEvalvid_Sink]
$ns attach-agent $d1 $null1
$ns connect $udp1 $null1
$null1 set_filename rd_a01
set original_file_name st_a01
set trace_file_name video1.dat
set original_file_id [open $original_file_name r]
set trace_file_id [open $trace_file_name w]
set pre_time 0
while {[eof $original_file_id] == 0} {
    gets $original_file_id current_line
    scan $current_line "%d%s%d%d%f" no_ frametype_ length_ tmp1_ tmp2_
    set time [expr int(($tmp2_ - $pre_time)*1000000.0)]
    if { $frametype_ == "I" } {
        set type_v 1
        set prio_p 0
    }
    if { $frametype_ == "P" } {
        set type_v 2
        set prio_p 0
    }
    if { $frametype_ == "B" } {
        set type_v 3
        set prio_p 0
    }
    if { $frametype_ == "H" } {
```

无线网络技术

```
            set type_v 1
            set prio_p 0
        }
        puts $trace_file_id "$time $length_ $type_v $prio_p $max_fragmented_size"
        set pre_time $tmp2_
}
close $original_file_id
close $trace_file_id
set end_sim_time $tmp2_
puts "$end_sim_time"
set trace_file [new Tracefile]
$trace_file filename $trace_file_name
set video1 [new Application/Traffic/myEvalvid]
$video1 attach-agent $udp1
$video1 attach-tracefile $trace_file

proc finish {} {
        global ns nd
        $ns flush-trace
        close $nd
        exit 0
}
$ns at 0.0 "$video1 start"
$ns at $end_sim_time "$video1 stop"
$ns at [expr $end_sim_time + 1.0] "$null1 closefile"
$ns at [expr $end_sim_time + 1.0] "finish"
$ns run
```

根据上述脚本代码，可按如下操作完成实验：

①使用“**Ctrl+Alt+T**”快捷键打开终端，并用“**cd /home/wiotlab/WIoTLab/exps/lab2**”命令切换路径，接着使用“**touch lab2_lan.tcl**”命令创建脚本文件，将上述代码抄录至该文件中；

②执行视频处理流程的①-③步(数据文件名与步骤中一致)，准备仿真所需的发送数据。假设 YUV 视频文件为 **foreman_qcif.yuv**；

③执行“**ns lab2_lan.tcl**”命令运行脚本，可获得仿真实验结果数据，其中与视频相关的数据文件为 **sd_a01** 和 **rd_a01**；

④执行视频处理流程的④-⑤步，得到视频 PSNR，也可使用 YUV 工具查看传输后的结果。

需注意，上述操作所使用视频、工具(使用 **chmod** 命令增加执行权限)和脚本代码位于相同的文件夹中。

(2)802.11e 无线视频传输

无线网络技术

该实验建立了简单的无线网络，包含发送和接收节点各 1 个，具体代码如下所示。

```
proc getopt {argc argv} {
      global opt
            lappend optlist nn
            for {set i 0} {$i < $argc} {incr i} {
           set opt($i) [lindex $argv $i]
      }
}
#1 mapping type (0: 802.11e, 1: static mapping, 2: dynamic mapping);
#2 number of voice flows (AC_3) ;
#3 number of video flows (AC_2) ;
#4 number of TCP flows      (AC_1) ;
#5 number of CBR flows      (AC_0) ;

getopt $argc $argv
set packetSize      1500
set max_fragmented_size      1024
set val(chan)                  Channel/WirelessChannel        ;# channel type
set val(prop)                  Propagation/TwoRayGround     ;# radio-propagation model
set val(netif)                Phy/WirelessPhy                ;# network interface type
set val(mac)                   Mac/802_11e                      ;# MAC type
set val(ifq)                   Queue/DTail/PriQ              ;# interface queue type
set val(ll)                    LL                                  ;# link layer type
set val(ant)                   Antenna/OmniAntenna          ;# antenna model
set val(ifqlen)               50                                ;# max packet in ifq
set val(rp)           AODV
set opt(choice)           $opt(0)
set opt(voiceflow)    $opt(1)
set opt(videoflow)     $opt(2)
set opt(TCPflow)$opt(3)
set opt(CBRflow)      $opt(4)
Mac/802_11e set dataRate_                1.0e6                ;# 1Mbps
Mac/802_11e set basicRate_               1.0e6                ;# 1Mbps
set ns [new Simulator]
set f [open lab2.tr w]
$ns trace-all $f
set topo             [new Topography]
$topo load_flatgrid 500 500
create-god 2
set chan [new $val(chan)]
$ns node-config -adhocRouting $val(rp) \
                        -llType $val(ll) \
                        -macType $val(mac) \
                        -ifqType $val(ifq) \
```

无线网络技术

```
                        -ifqLen $val(ifqlen) \
                        -antType $val(ant) \
                        -propType $val(prop) \
                        -phyType $val(netif) \
                        -channel $chan \
                        -topoInstance $topo \
                        -agentTrace OFF \
                        -routerTrace OFF \
                        -macTrace OFF \
                        -movementTrace OFF

for {set i 0} {$i < 2} {incr i} {
            set node_($i) [$ns node]
            $node_($i) random-motion 0
}
$node_(0) set X_ 30.0
$node_(0) set Y_ 30.0
$node_(0) set Z_ 0.0
$node_(1) set X_ 200.0
$node_(1) set Y_ 30.0
$node_(1) set Z_ 0.0

#1st priority traffic (VoIP)
for {set i 0} {$i < $opt(voiceflow) } {incr i} {
      set udpA_($i) [new Agent/UDP]
      $udpA_($i) set prio_ 0
      $ns attach-agent $node_(0) $udpA_($i)
      set nullA_($i) [new Agent/Null]
      $ns attach-agent $node_(1) $nullA_($i)
      $ns connect $udpA_($i) $nullA_($i)
      set voip_($i) [new Application/Traffic/CBR]
      $voip_($i) attach-agent $udpA_($i)
      $voip_($i) set packet_size_ 160
      $voip_($i) set rate_ 64k
      $voip_($i) set random_ false
      $ns at 5.0 "$voip_($i) start"
      $ns at 50.0 "$voip_($i) stop"
}

#2nd priority traffic (Video)
for {set i 0} {$i < $opt(videoflow) } {incr i} {
      set udp($i) [new Agent/my_UDP]
      $ns attach-agent $node_(0) $udp($i)
      $udp($i) set packetSize_ $packetSize
```

无线网络技术

```
$udp($i) set_filename sd_a01
set null($i) [new Agent/myEvalvid_Sink]
$ns attach-agent $node_(1) $null($i)
$ns connect $udp($i) $null($i)
$null($i) set_filename rd_a01
set original_file_name($i) st_a01
set trace_file_name($i) video($i).dat
set original_file_id($i) [open $original_file_name($i) r]
set trace_file_id($i) [open $trace_file_name($i) w]
set pre_time 0
set totalByte_I 0
set totalByte_P 0
set totalByte_B 0
set totalPkt_I 0
set totalPkt_P 0
set totalPkt_B 0
while {[eof $original_file_id($i)] == 0} {
       gets $original_file_id($i) current_line
       scan $current_line "%d%s%d%d%f" no_ frametype_ length_ tmp1_ tmp2_
       set time [expr int(($tmp2_ - $pre_time)*1000000.0)]
       if { $frametype_ == "I" } {
      set type_v 1
      set prio_p 1
      set totalByte_I    [expr int($totalByte_I + $length_)]
      set totalPkt_I [expr int($totalPkt_I + $tmp1_)]
}
       if { $frametype_ == "P" } {
      set type_v 2
      set prio_p 1
      set totalByte_P    [expr int($totalByte_P + $length_)]
        set totalPkt_P [expr int($totalPkt_P + $tmp1_)]
       }
       if { $frametype_ == "B" } {
      set type_v 3
      set prio_p 1
      set    totalByte_B    [expr int($totalByte_B + $length_)]
      set totalPkt_B [expr int($totalPkt_B + $tmp1_)]
       }
       if { $frametype_ == "H" } {
      set type_v 1
      set prio_p 1
      set totalByte_I    [expr int($totalByte_I + $length_)]
      set totalPkt_I [expr int($totalPkt_I + $tmp1_)]
       }
```

无线网络技术

```
            puts    $trace_file_id($i) "$time $length_ $type_v $prio_p $max_fragmented_size"
            set pre_time $tmp2_
        }
        set totalPkt    [expr int($totalPkt_I+$totalPkt_P+$totalPkt_B)]
        set totalByte    [expr int($totalByte_I+$totalByte_P+$totalByte_B)]
        close $original_file_id($i)
        close $trace_file_id($i)
        set end_sim_time $tmp2_
        puts "end_sim_time: $end_sim_time"
        set trace_file($i) [new Tracefile]
        $trace_file($i) filename $trace_file_name($i)
        set video($i) [new Application/Traffic/myEvalvid]
        $video($i) attach-agent $udp($i)
        $video($i) attach-tracefile $trace_file($i)
        $ns at [expr 0.1] "$video($i) start"
        $ns at [expr 50.0] "$video($i) stop"
        $ns at [expr 50.0] "$null($i) closefile"
        $ns at [expr 50.0] "$null($i) printstatus"
}
#3rd priority traffic (CBR)
for {set i 0} {$i < $opt(CBRflow) } {incr i} {
        set udpB_($i) [new Agent/UDP]
        $udpB_($i) set prio_ 2
        $ns attach-agent $node_(0) $udpB_($i)
        set nullB_($i) [new Agent/Null]
        $ns attach-agent $node_(1) $nullB_($i)
        $ns connect $udpB_($i) $nullB_($i)
        set cbr_($i) [new Application/Traffic/CBR]
        $cbr_($i) attach-agent $udpB_($i)
        $cbr_($i) set packet_size_ 200
        $cbr_($i) set rate_ 125k
        $cbr_($i) set random_ false
        $ns at 20.0 "$cbr_($i) start"
        $ns at 35.0 "$cbr_($i) stop"
}
#4th priority traffic (TCP)
for {set i 0} {$i < $opt(TCPflow) } {incr i} {
        set tcp($i) [new Agent/TCP]
        $tcp($i) set prio_ 3
        $ns attach-agent $node_(0) $tcp($i)
        set sink($i) [new Agent/TCPSink]
        $ns attach-agent $node_(1) $sink($i)
        $ns connect $tcp($i) $sink($i)
        set ftp($i) [new Application/FTP]
```

无线网络技术

```
        $ftp($i) set type_ FTP
        $ftp($i) attach-agent $tcp($i)
        $ns at 15.0 "$ftp($i) start"
        $ns at 30.0 "$ftp($i) stop"
}

set n0_ifq [$node_(0) set ifq_(0)]
$n0_ifq set choice $opt(choice)
$n0_ifq set threshold1 10 # low threshold of queue length
$n0_ifq set threshold2 40 # high threshold of queue length
$n0_ifq set prob0 0         # Prob_I
$n0_ifq set prob1 0.6       # Prob_P
$n0_ifq set prob2 0.9       # Prob_B
for {set i 0} {$i < 2} {incr i} {
        $ns initial_node_pos $node_($i) 30
        $ns at 50.0 "$node_($i) reset";
}
$ns at 50.0 "finish"
$ns at 50.1 "puts \"NS EXITING...\"; $ns halt"
proc finish {} {
        global ns f
        $ns flush-trace
        close $f
}
puts "Starting Simulation..."
$ns run
```

在上述代码中，MAC 层采用了 802.11e 协议，且配置为默认。根据上述脚本代码，可按如下操作完成实验：

①使用“**Ctrl+Alt+T**”快捷键打开终端，~~并用“**cd /home/wiotlab/WIoTLab/exps/lab2**”命令切换路径~~，接着使用“**touch 802_11e.tcl**”命令创建脚本文件，将上述代码抄录至该文件中；

②执行视频处理流程的①-③步(数据文件名与步骤中一致)，准备仿真所需的发送数据。假设 YUV 视频文件为 **foreman_qcif.yuv**；

③执行“**ns 802_11e.tcl**”命令运行脚本，可获得仿真实验结果数据，其中与视频相关的数据文件为 **sd_a01** 和 **rd_a01**；

④执行视频处理流程的④-⑤步，得到视频 PSNR，也可使用 YUV 工具查看传输后的结果。

同第 1 个实验，上述操作所使用视频、工具(使用 **chmod** 命令增加执行权限)和脚本代码位于相同的文件夹中。

无线网络技术

2.4 扩展练习*

本章实验展示了两个简单网络场景的视频传输仿真过程，读者可参照上述例子自行设计自己的网络场景并进行仿真分析，也可深入探索 IEEE 802.11e 协议的性能。除此之外，视频传输问题的研究也是近年来的热点，受到广泛关注，感兴趣的读者可自行调研基于 HTTP 的动态自适应视频流媒体(Dynamic Adaptive Streaming over HTTP, DASH)[2]和自适应帧率(Adaptive Frame Rate, AFR)等前沿技术，并进行深入探索。

参考文献

[1] F. Roijers, H. Berg, X. Fan, et al. A Performance Study on Service Integration in IEEE 802.11E Wireless LANs. Computer Communications, 2006, 29: 2621-2633.

[2] X. Costa-Perez, D. Camps-Mur, T. Sashihara. Analysis of the Integration of IEEE 802.11e Capabilities in Battery Limited Mobile Devices. IEEE Wireless Communications, 2005, 12(6): 26-32.

[3] P. Xavier, C. Daniel, V. Albert. On Distributed Power Saving Mechanisms of Wireless LANs 802.11e U-APSD vs 802.11 Power Save Mode. Computer Networks, 2007, 51: 2326-2344.

[4] J. Klaue, B. Rathke, A. Wolisz. EvalVid - A Framework for Video Transmission and Quality Evaluation. In Proc. International Conference on Modelling Techniques and Tools for Computer Performance Evaluation, Sep. 2003, Urbana, USA.

[5] D. He, C. Q. Shen. Simulation Study of IEEE 802.11e EDCF. In Proc. VTC 2003-Spring, April 2003, Jeju, South Korea.

[6] Z. Meng, T. Wang, Y. Shen, et al. Enabling High Quality Real-Time Communications with Adaptive Frame-Rate. In Proc. NSDI, April 2023, Boston, USA.

无线网络技术

[2] https://dashif.org/